

第 56 屆全國技能競賽網頁技術青少年組

競賽公開試題

職類名稱：網頁技術

共 14 頁

競賽時間：4 小時

(註：本公開試題在競賽時得有百分之三十之調整)

選手注意事項：

1. 考試時間為 4 小時，但時間有異動時，以大會規定另行公告。
 2. 選手請自行安裝 XAMPP 套件，設定工作環境，包含 web server 及資料庫。
 3. 每一「工作崗位」皆有一個隨身碟，內有一個「參考資料」資料夾，作答時可參考使用此資料夾內的資料。
 4. 請每位選手依 **web+「考場工作崗位編號」** 在桌面建立一個資料夾(例如 web01, web02)，用來存放要被評分資料。
 5. 網站首頁請設定為 index.htm，或 index.php。
 6. 作答期間，若是資料庫或是網頁需設定帳號密碼時，請依照下列方式設定：
 - 網頁帳號設定為 **admin**、密碼設定為 **1234**
 - 資料庫使用者帳號設定為崗位編號(webXX)，密碼設定為 1234，資料庫設定為崗位編號_db(webXX_db)
- 若未依規定扣總分 5 分。**
7. 考試相關資料於考完後，全部收回，包括考場提供的空白紙，試題、評分表、資料片等。
 8. 每一「評分項目」的前面，皆有標明「主 or 客」代表「主觀評分 or 客觀評分」。
 9. 若有同分時決定名次的先後排名方式如下：
 - (1) 先依作答交卷時間之先後順序，來比較排名，交卷時間較早者，排名先。離競賽時間結束前 10 分到競賽結束，交卷時間比序等級視同一樣。
 - (2) 若交卷時間比序等級一樣時，則依序以評分表中的評分項次得分來比較排名，從第 1 項次開始比較到最後一項次，得分高者，排名先。
 - (3) 最後，新增主觀評分項目「網站設計整體性」項次，由所有裁判人員共同評分，依得分高低比較，來比較名次的先後。
 10. 其他注意事項：
 - (a) 請參考評分標準表作答。
 - (b) 除非題目中特別說明，否則表單無須檢查資料是否有誤，或是有沒有填寫。
 - (c) 除非題目中特別說明要評量美工之設計分數，否則評分重點在於功能是否完成，與畫面之美工無關。
 - (d) 請隨時將設計好之網頁、圖片及資料庫等儲存至隨身碟，以避免當機等意外情形發生。
 - (e) 比賽結束前請將作答之整個資料夾複製到隨身碟中。若成績產生爭議時，將以此隨身碟中之結果為依據。
 - (f) 比賽結束時間一到，選手不可以再操作電腦，若因此未能將最新之答案存入隨身碟及桌面評分資料夾內，致使成績產生問題，選手須自負一切責任。
 - (g) 比賽結束時，請勿關機，以利評分作業之進行。

● 模組 1、「FunTech 社群網站」視覺與網頁介面設計

請幫忙協助製作「FunTech 社群網站」視覺與網頁介面設計，內容如：LOGO 標誌、ICON、網頁介面等設計。

● 模組 2、FunTech 社群網站 平台翻新與系統建置

FunTech 是一個專為青少年打造的教育與娛樂社群網站，提供多元資訊頁面、會員登入系統及簡易遊戲平台。此平台於先前已完成基礎網站架構與部分功能建置，並獲得不錯的發展成果。

隨著平台逐漸成長，FunTech 團隊希望對網站進行全面升級與重新設計，以提升整體使用體驗並強化平台功能。因此，本次將以 FunTech 為主題，重新規劃與建置網站系統。選手需在保留平台核心理念的前提下，重新設計網站版面與功能架構。

本試題將請選手依據提供之示意圖與素材，完成 FunTech 社群網站的版面設計與系統功能建置，並依規格實作指定之網站模組，使平台能夠提供完整的遊戲展示與管理機制。

項目 1：首頁版面設計與個人頁面功能開發

FunTech 平台首頁為網站主要入口，需呈現「文章列表」與「通知/公告」。首頁必須包含以下區塊：導覽列 (Header)、文章列表區、通知列表區。版面可由選手自由設計，但必須提供指定的 DOM 結構與 CSS selector，以利自動化測試進行功能驗證。

1 首頁

1.1 首頁需有一個可辨識的根容器，包住整個首頁內容。(#home)

1.2 頁首導覽列 Header

1.2.1 頁面頂端需有頁首導覽列容器。(header.site-header)

1.2.2 頁首需包含

1.2.2.1 FunTech Logo，並可點擊回到首頁。

(.site-header .brand a.brand-link)

1.2.2.2 主要導覽列容器。(.site-header nav.main-nav)

1.2.2.2.1 「首頁」連結，用以返回網站首頁。(.site-header nav.main-nav a.home-link)

1.2.2.2.2 「遊戲」連結，用以進入平台的遊戲列表頁面。

(`.site-header nav.main-nav a.games-link`)

1.2.2.2.3 「好友」連結，用以進入好友列表頁面。(`.site-header`

`nav.main-nav a.friends-link`)

1.2.2.3 「使用者區域」容器。(`.site-header .user-area`)

1.2.2.3.1 未登入狀態時，使用者區域需顯示「登入」連結。

(`.user-area a.login-link`) 以及「註冊」連結。

(`.user-area a.register-link`)

1.2.2.3.2 已登入狀態時，使用者區域需顯示使用者名稱或頭像

區塊。(`.user-area .user-badge`)、個人頁面入口

(`.user-area a.profile-link`) 以及「登出」按鈕或連結。

(`.user-area a.logout-link`)

1.3 文章列表 Articles

1.3.1 首頁需有文章區塊容器。(`section.articles`)

1.3.2 文章列表需呈現多筆文章，每筆文章需使用語意化元素呈現，且

每筆文章節點需套用 class。(`section.articles`

`article.article-item`)

1.3.3 每筆文章需有

1.3.3.1 「文章標題」。(`.article-item .article-title`)

1.3.3.2 「發佈日期」。(`.article-item time.article-date`)

1.3.3.3 「文章摘要」。(`.article-item .article-excerpt`)

1.3.3.4 「閱讀更多」連結，點擊可前往該文章內容頁。

(`.article-item a.article-readmore`)

1.4 通知 / 公告 Notifications

1.4.1 首頁需有通知 / 公告區塊容器。(`aside.notifications`)

1.4.2 通知列表需呈現多則通知，每則通知需為一個可重複的項目節點。

(`aside.notifications .notification-item`)

1.4.3 每則通知需顯示

1.4.3.1 「通知標題」。(`.notification-item .notification-title`)

1.4.3.2 「發佈日期」。(`.notification-item`

`time.notification-date`)

2 文章內容頁

2.1 文章內容頁需有一個可辨識的根容器包住整個頁面內容。(`#article`)

2.2 文章標題區塊 Article Header

2.2.1 文章需包含標題區塊。(#article header.article-header)

2.2.2 標題區塊需顯示

2.2.2.1 「文章標題」。(.article-header h1.article-title)

2.2.2.2 「文章發布日期」。(.article-header time.article-date)

2.3 文章內容 Article Body

2.3.1 文章需包含內容區塊。(#article section.article-body)

3 個人頁面

3.1 個人頁面需有一個可辨識的根容器包住整個頁面內容。(#profile-page)

3.2 使用者資訊區

3.2.1 頁面需包含使用者資訊區塊。(section.profile-header)

3.2.2 使用者資訊區需顯示

3.2.2.1 使用者頭像 (可為預設圖片)。(.profile-header
img.profile-avatar)

3.2.2.2 使用者名稱。(.profile-header .profile-username)

3.2.2.3 簡介文字 (若無簡介則顯示「尚未填寫自我介紹」)
(.profile-header .profile-bio)

3.2.3 頭像修改

3.2.3.1 當使用者點擊頭像圖片時，需觸發圖片上傳功能。

3.2.3.2 系統需開啟檔案選擇視窗，供使用者選擇圖片檔案。

3.2.3.3 使用者選擇圖片後，系統需更新該使用者的頭像。

3.2.3.4 更新成功後，頁面上的頭像需即時更新顯示。

3.2.3.5 若上傳失敗，需透過 alert 顯示錯誤訊息「頭像上傳失敗」。

3.2.4 簡介文字修改

3.2.4.1 當使用者點擊簡介文字時，簡介區需轉為可編輯狀態。

3.2.4.2 系統需顯示簡介輸入欄位。(.profile-header
textarea.profile-bio-input)

3.2.4.3 使用者修改完成後，可透過按下 Enter 鍵儲存：

3.2.4.4 儲存成功後，頁面需即時更新簡介文字顯示。

3.2.4.5 若儲存失敗，需透過 alert 顯示錯誤訊息「簡介更新失敗」。

3.3 發表文章功能

3.3.1 當使用者已登入且瀏覽自己的個人頁面時，頁面需包含發表文章的連結。(`a.new-post-link`)，當點擊後將會連結至文章發布區。

3.3.2 文章發布區需包含文章表單容器。(`form.article-create-form`)

3.3.3 文章表單需包含

3.3.3.1 文章標題輸入欄位。(`.article-create-form input.article-title-input`)

3.3.3.2 文章內容輸入欄位。(`.article-create-form textarea.article-content-input`)

3.3.3.3 發布文章按鈕。(`.article-create-form button.article-submit-button`)

3.3.4 當發表文章成功時，將會透過 `alert` 顯示「發表成功」，並且連結至新文章的內容頁面。

3.4 使用者文章區

3.4.1 此區塊需顯示該使用者所發表的文章列表，頁面需包含使用者文章區塊。(`section.profile-articles`)

3.4.2 每篇文章需為一個文章項目。(`.profile-articles .article-item`)

3.4.3 每篇文章需顯示

3.4.3.1 文章標題。(`.article-item .article-title`)

3.4.3.2 發佈日期。(`.article-item time.article-date`)

3.4.3.3 閱讀文章連結。(`.article-item a.article-readmore`)

3.4.3.4 點擊文章連結需導向該文章內容頁。

3.4.4 當該使用者尚未發表任何文章時，需顯示提示文字「目前尚無文章」。(`.profile-articles .empty-article-message`)

項目 2：會員登入與註冊系統

1 登入頁面 Login

- 1.1 登入頁面需包含登入表單容器。(`form.login-form`)
- 1.2 登入表單需包含以下欄位：
 - 1.2.1 使用者帳號輸入欄位。(`.login-form input.username-input`)
 - 1.2.2 使用者密碼輸入欄位。(`.login-form input.password-input`)
 - 1.2.3 登入送出按鈕。(`.login-form button.login-submit-button`)
- 1.3 使用者輸入帳號與密碼後，點擊登入按鈕應送出登入請求並進行驗證。

2 註冊頁面 Register

- 2.1 註冊頁面需包含註冊表單容器。(`form.register-form`)
- 2.2 註冊表單需包含以下欄位：
 - 2.2.1 使用者帳號輸入欄位。(`.register-form input.username-input`)
 - 2.2.2 使用者電子郵件欄位。(`.register-form input.email-input`)
 - 2.2.3 使用者密碼輸入欄位。(`.register-form input.password-input`)
 - 2.2.4 確認密碼輸入欄位。(`.register-form input.password-confirm-input`)
 - 2.2.5 註冊送出按鈕。(`.register-form button.register-submit`)
- 2.3 使用者填寫資料並送出後，系統需建立新帳號。
- 2.4 註冊成功後，系統需導向登入頁面。
- 2.5 若帳號已存在或資料格式錯誤，需透過 `alert` 顯示相應的錯誤訊息

3 登出 Logout

項目 3：好友系統 (Friend System)

在 FunTech 平台中，使用者不僅可以建立自己的個人頁面，也可以與其他使用者建立社交連結，為了讓平台具有基本的社群互動能力，本項目要求設計並實作一個好友系統 (Friend System)。

好友系統將允許使用者：

- 搜尋其他使用者
- 查看好友的個人頁面
- 發送好友邀請
- 接受或拒絕好友申請
- 查看好友列表
- 移除好友

1 好友頁面

1.1 所有登入使用者必須能夠透過導覽列的「好友」連結，進入好友列表頁面，進入後至少包含以下區塊：

1.1.1 好友頁面需包含好友頁主容器。(`#friends-page`)，包含以下各個區塊：

1.1.1.1 使用者搜尋功能區塊

(`#friends-page.friend-search-section`)

1.1.1.2 我的好友列表區塊(`#friends-page.friend-list-section`)

1.1.1.3 我收到的好友邀請區塊

(`#friends-page.incoming-requests-section`)

1.1.1.4 我送出的好友申請區塊

(`#friends-page.sent-requests-section`)

2 搜尋使用者功能 (`#friends-page.friend-search-section`)

2.1 搜尋區塊需包含搜尋表單容器。(`form.friend-search-form`)

2.2 搜尋表單需包含以下欄位：

2.2.1 搜尋輸入欄位。(`.friend-search-form input.search-input`)

2.2.2 搜尋按鈕。(`.friend-search-form button.search-submit-button`)

2.3 使用者輸入關鍵字並點擊搜尋按鈕後，系統需向後端送出搜尋請求，並更新搜尋結果清單。

2.4 搜尋結果需顯示於搜尋結果清單容器。

- (`.friend-search-section .search-result-list`)
- 2.5 每一筆搜尋結果需為一個結果項目。
 - (`.search-result-list .search-result-item`)
- 2.6 每一筆搜尋結果項目需顯示：
 - 2.6.1 使用者名稱。(`.search-result-item .result-username`)
 - 2.6.2 前往個人頁面連結。(`.search-result-item a.view-profile-link`)
- 3 我的好友列表區塊
 - 3.1 好友列表區塊需顯示標題「好友列表」。
 - (`.friend-list-section .section-title`)
 - 3.2 好友列表需顯示目前使用者的所有好友，每一位好友需為一個可重複的列表項目。(`.friend-list-section .friend-item`)
 - 3.3 每個好友項目需顯示
 - 3.3.1 好友頭像。(`.friend-item img.friend-avatar`)
 - 3.3.2 好友名稱。(`.friend-item .friend-name`)
 - 3.4 點擊好友項目時，需導向該好友的個人頁面。
- 4 我收到的好友邀請區塊
 - 4.1 此區塊需顯示標題「收到的好友申請」。
 - (`.incoming-requests-section .section-title`)
 - 4.2 此區塊需顯示其他使用者向目前使用者發送的好友申請。
 - 4.3 每一筆好友申請需為一個可重複的列表項目。
 - (`.incoming-requests-section .request-item`)
 - 4.4 每個好友申請項目需顯示
 - 4.4.1 申請者頭像。(`.request-item img.request-avatar`)
 - 4.4.2 申請者名稱。(`.request-item .request-username`)
 - 4.4.3 接受好友按鈕。(`.request-item button.accept-request-button`)
 - 4.4.4 拒絕好友按鈕。(`.request-item button.reject-request-button`)
 - 4.5 當使用者點擊「接受」按鈕時，系統需建立好友關係，並將該使用者加入好友列表。
 - 4.6 當使用者點擊「拒絕」按鈕時，該好友申請需從列表中移除。
- 5 我送出的好友申請區塊
 - 5.1 此區塊需顯示標題「發送的好友申請」。
 - (`.sent-requests-section .section-title`)
 - 5.2 此區塊需顯示目前使用者送出的好友申請。

- 5.3 每一筆好友申請需為一個可重複的列表項目。
(`.sent-requests-section .request-item`)
- 5.4 每個好友申請項目需顯示
 - 5.4.1 對方使用者頭像。(`.request-item img.request-avatar`)
 - 5.4.2 對方使用者名稱。(`.request-item .request-username`)
 - 5.4.3 取消申請按鈕。(`.request-item button.cancel-request-button`)
- 5.5 當使用者點擊「取消」按鈕時，系統需取消該好友申請，並從列表中移除該項目。
- 6 好友的個人頁面
 - 6.1 好友個人頁面需包含主容器。(`#profile-page`)，包含
 - 6.1.1 使用者資訊區塊。(`#profile-page .profile-header`)，包含
 - 6.1.1.1 使用者名稱。(`.profile-header .profile-username`)
 - 6.1.1.2 圖片 (可為預設圖片)。(`.profile-header img.profile-avatar`)
 - 6.1.1.3 簡介文字 (若無簡介則顯示「尚未填寫自我介紹」)。(`.profile-header .profile-bio`)
 - 6.1.2 內容展示區塊。(`#profile-page .profile-content`)
 - 6.1.2.1 需有文章區塊容器。(`section.articles`)
 - 6.1.2.2 需呈現多筆文章，每筆文章需使用語意化元素呈現，且每筆文章節點需套用 class。(`section.articles article.article-item`)
 - 6.1.2.3 每筆文章需有
 - 6.1.2.3.1 「文章標題」。(`.article-item .article-title`)
 - 6.1.2.3.2 「發佈日期」。(`.article-item time.article-date`)
 - 6.1.2.3.3 「文章摘要」。(`.article-item .article-excerpt`)
 - 6.1.2.3.4 「閱讀更多」連結，點擊可前往該文章內容頁。(`.article-item a.article-readmore`)
 - 6.1.3 好友互動操作區塊。(`#profile-page .profile-friend-actions`)
 - 6.1.3.1 此區塊需要根據目前使用者與對方的好友狀態有不同的呈現方式

項目 4：遊戲頁面 Games

此項目將實作平台中的「遊戲系統」，包含遊戲列表頁面、遊戲內容頁面以及排行榜資料整合，平台會提供多個可遊玩的遊戲，使用者可以在遊戲列表中選擇遊戲並開始遊玩，同時查看該遊戲的排行榜，系統中的遊戲資料會儲存在資料庫的 `games` 資料表中，網站需透過此資料表取得遊戲列表資訊，並依據每個遊戲的設定載入遊戲內容與排行榜資料。

為了讓選手專注於前端畫面整合與資料處理，本題目所需的素材皆會由主辦單位提供，包含：

- `games` 資料表結構的 SQL 建立語法
- 每個遊戲的專案資料夾
 - 提供排行榜資料的 API

選手不需要自行撰寫遊戲程式或分數 API，僅需依據題目規格：

- 從資料庫取得遊戲列表資訊
- 讀取各遊戲資料夾中的 `game.json` 設定
- 載入遊戲入口網址
- 呼叫排行榜 API 並將資料顯示於頁面中

在 `/games` 資料夾下每一個資料夾都是一個遊戲，例如以下結構：

```
/
├── index.html
├── games/
│   ├── 1/
│   │   └── game.json
│   ├── 2/
│   │   └── game.json
│   └── 3/
│       └── game.json
```

其中必定包含一個 `game.json`。

game.json 包含以下資料：

```
{
  "entry": {
    "url": "http://localhost/gameA/index.php"
  },
  "score": {
    "pullUrl": "http://localhost/gameA/api/pull_score.php",
    "columns": ["玩家名稱", "分數"]
  }
}
```

欄位說明:

- entry.url: 遊戲的入口網址
- score.pullUrl: 取得遊戲分數資料的 API 網址，可以從此網址取得該遊戲的分數資料
- score.columns: 此欄位為排行榜資料的欄位名稱列表，也是 score.pullUrl 回傳資料的欄位內容，可以依據此欄位顯示排行榜資訊

分數 API 回傳資料格式

score.pullUrl 回傳的 JSON 資料需為 陣列格式，每一筆資料代表一位玩家的分數紀錄。

```
[
  {
    "玩家名稱": "玩家 1",
    "分數": 95000
  },
  {
    "玩家名稱": "玩家 2",
    "分數": 89000
  },
  {
    "玩家名稱": "玩家 3",
    "分數": 82500
  }
]
```

```
    },  
    {  
      "玩家名稱": "玩家 4",  
      "分數": 82000  
    },  
    {  
      "玩家名稱": "玩家 5",  
      "分數": 70500  
    }  
  ]  
}
```

1 遊戲列表頁面

1.1 遊戲列表頁需有一個可辨識的根容器，包住整個頁面內容。(`#games`)

1.2 遊戲列表

1.2.1 遊戲列表頁需包含遊戲列表容器。(`#games section.game-list`)

1.2.2 遊戲列表需呈現多個遊戲，每個遊戲需為一個可重複的項目節點。
(`.game-list .game-item`)

1.2.3 每個遊戲項目需顯示

1.2.3.1 遊戲封面圖片。(`.game-item img.game-cover`)

1.2.3.2 遊戲名稱。(`.game-item .game-title`)

1.2.3.3 遊戲簡短介紹。(`.game-item .game-description`)

1.2.3.4 開始遊戲連結。(`.game-item a.play-game-link`)

1.2.3.5 點擊「開始遊戲」連結需導向遊戲內容頁。

2 遊戲內容頁面

2.1 遊戲內容頁需有一個可辨識的根容器包住整個頁面內容。

(`#game-play`)

2.2 需顯示目前遊戲名稱 (`.current-game-title`)

2.3 遊戲區域

2.3.1 頁面需包含遊戲區域容器。(`#game-play section.game-area`)

2.3.2 遊戲內容需透過 `iframe` 載入。(`section.game-area
iframe.game-frame`)

2.4 排行榜區域

2.4.1 遊戲內容頁需包含排行榜區塊。(`aside.game-leaderboard`)

2.4.2 排行榜區塊需顯示標題。(`.game-leaderboard .leaderboard-title`)

2.4.3 排行榜需呈現多筆玩家成績資料。

(`.game-leaderboard .leaderboard-item`)

2.4.4 每筆排行榜資料需顯示名次 (`.leaderboard-item .player-rank`)

以及從分數 API 當中拿回的所有資料欄位(例如玩家名稱、分數)

2.4.5 若排行榜資料為空，需顯示「目前尚無分數紀錄」。

項目 5 : Web API 開發

試題描述與工作說明：

競賽當天公布。

API 基本需求

- 所有 API 回應使用 JSON 格式
- 需實作基本的錯誤處理

選手提示

1. 在瀏覽器上看到格式不同不影響結果，只要資料內容正確即可。
2. API 所呈現的資料必須與系統同步，也就是說當網站後台存在 N 筆資料時，API 也需要回傳 N 筆資料。